

Problema Lab 3 : Introducere în ORM și Connection Pooling

Obiectiv

Refactorizați aplicația din Lab 1 pentru a utiliza un framework ORM (Hibernate pentru Java sau Entity Framework Core pentru C#) și implementați connection pooling pentru a îmbunătăți performanța aplicației și mentenabilitatea codului.

Obiective de Învățare

- Înțelegerea beneficiilor și compromisurilor utilizării framework-urilor ORM
- Învățarea mapării tabelor bazei de date la clase orientate pe obiecte
- Implementarea și configurarea connection pooling-ului
- Măsurarea și compararea performanței cu și fără connection pooling
- Înțelegerea gestionării corecte a resurselor în contextul ORM

Cerințe

1. Migrare la ORM

Refactorizați aplicația din Lab 1 pentru a utiliza:

- **Java:** Hibernate ORM cu adnotări JPA
- **C#:** Entity Framework Core

Clase Entitate:

- Creați clase entitate pentru ambele tabele (părinte și copil)
- Utilizați adnotări/atribute adecvate pentru:
 - Chei primare
 - Relații de cheie străină (1-la-N)
 - Mapări de coloane
 - Proprietăți de navigare în relații

Înlocuirea Interogărilor SQL:

- Eliminați toate interogările SQL manuale din Lab 1
- Utilizați API-urile de interogare ORM:
 - Java: JPQL sau Criteria API
 - C#: Interogări LINQ
- Mențineți toată funcționalitatea CRUD din Lab 1

2. Implementarea Connection Pooling

Java - HikariCP:

```
// Adăugați în configurarea dvs.
HikariConfig config = new HikariConfig();
config.setJdbcUrl("jdbc:postgresql://localhost:5432/yourdb");
config.setUsername("user");
config.setPassword("password");
config.setMaximumPoolSize(10);
config.setMinimumIdle(5);
config.setConnectionTimeout(30000);

HikariDataSource dataSource = new HikariDataSource(config);
```

C# - Connection Pooling Integrat:

```
// Configurați în DbContext
protected override void OnConfiguring(DbContextOptionsBuilder options)
    => options.UseNpgsql(
        "Host=localhost;Database=yourdb;Username=user;Password=pass;Maximum Pool Size=10;Minimum Pool Si
```

3. Exercițiu de Măsurare a Performanței

Sarcina A: Overhead-ul Creării Conexiunilor

Creați un test care măsoară timpul pentru:

1. Crearea a 100 de conexiuni la baza de date **fără** pooling (creați conexiune nouă de fiecare dată)
2. Obținerea a 100 de conexiuni **cu** pooling (din pool-ul de conexiuni)

Cerințe:

- Măsurați timpul total și timpul mediu per conexiune
- Afișați rezultatele în UI-ul aplicației sau în consolă
- Includeți rezultatele în raport cu analiză

Sarcina B: Detectarea Scurgerilor de Conexiuni

Demonstrați ce se întâmplă când conexiunile nu sunt închise corespunzător:

1. Creați un scenariu în care conexiunile sunt deschise dar nu închise
2. Arătați că pool-ul devine epuizat
3. Remediați problema cu gestionarea corectă a resurselor

4. Comparăție de Cod

Înainte (Lab 1 - JDBC/ADO.NET Pur):

```
// Exemplu de ce aveați
String sql = "SELECT * FROM employees WHERE department_id = ?";
PreparedStatement stmt = conn.prepareStatement(sql);
stmt.setInt(1, departmentId);
ResultSet rs = stmt.executeQuery();
```

```
// ... procesare manuală a rezultatelor
```

După (Lab 2 - ORM):

```
// Ce ar trebui să aveți acum
List<Employee> employees = entityManager
    .createQuery("SELECT e FROM Employee e WHERE e.department.id = :deptId", Employee.class)
    .setParameter("deptId", departmentId)
    .getResultList();
```

Includeți astfel de comparații în raport pentru a arăta reducerea codului boilerplate.

5. Cerințe Suplimentare

Lazy vs Eager Loading:

- Configurați relația părinte-copil să folosească lazy loading implicit
- Implementați cel puțin un scenariu unde aveți nevoie de eager loading
- Demonstrați diferența în interogările SQL generate

Gestionarea Tranzacțiilor:

- Utilizați gestionarea tranzacțiilor ORM pentru toate operațiile de scriere
- Arătați gestionarea corectă a tranzacțiilor în operațiile CRUD

Configurare:

- Externalizați detaliile conexiunii la baza de date într-un fișier de configurare
- Nu codificați hard credențialele în codul sursă

6. Livrabile

1. **Cod sursă refactorizat** cu implementare ORM
2. **Fișiere de configurare** (persistence.xml, hibernate.cfg.xml sau appsettings.json)
3. **Rezultate măsurări performanță** - capturi de ecran sau output consolă arătând:
 - Timpul de creare conexiuni fără pooling
 - Timpul de creare conexiuni cu pooling
 - Configurația pool-ului de conexiuni folosită
4. **Raport de comparație (1-3 pagini)** incluzând:
 - Comparație cod: înainte (JDBC/ADO.NET) vs după (ORM)
 - Analiza reducerii liniilor de cod
 - Rezultatele măsurărilor de performanță și interpretarea lor
 - Avantaje și dezavantaje ale ORM observate
 - Provocări întâmpinate în timpul migrării

7. Criterii de Evaluare (100 puncte)

- **Implementare ORM (40 puncte)**
 - Mapare corectă a entităților cu relații (20)

- Toate operațiile CRUD funcționează cu ORM (15)
- Utilizare corectă a limbajului de interogare ORM (5)
- **Connection Pooling (25 puncte)**
 - Configurare corectă (10)
 - Măsurători de performanță completate (10)
 - Analiza rezultatelor (5)
- **Calitatea Codului (20 puncte)**
 - Gestionare corectă a resurselor (10)
 - Configurare externalizată (5)
 - Organizarea codului (5)
- **Documentație (15 puncte)**
 - Completitudinea raportului de comparație (10)
 - Calitatea analizei performanței (5)

8. Puncte Bonus (+10)

- Implementați generarea schemei bazei de date din entități
- Adăugați logging pentru a arăta interogările SQL generate de ORM
- Comparați performanța interogărilor: SQL nativ vs SQL generat de ORM
- Implementați caching pentru entitățile accesate frecvent

Predare

- **Termen limită:** Următoarea sesiune de laborator (14 de zile)